

# Data Representation, Number Systems & Logic — Detailed Lecture Notes

JKSSB Computer Awareness

October 6, 2026

## 50.1 Bases and positional notation

Digital circuits commonly use two reliably distinguishable states, written as 0 and 1. A number system specifies a base (also called a radix) and the digits allowed in each position. The base determines the place values.

**Definition (Number-system base).** *The number of distinct digit symbols in a positional number system, with place values given by powers of that base.*

| Name        | Base | Allowed digits | Example   |
|-------------|------|----------------|-----------|
| Binary      | 2    | 0–1            | $1011_2$  |
| Octal       | 8    | 0–7            | $27_8$    |
| Decimal     | 10   | 0–9            | $45_{10}$ |
| Hexadecimal | 16   | 0–9, A–F       | $2F_{16}$ |

The subscript identifies the base. In hexadecimal, A represents 10, B 11, C 12, D 13, E 14, and F 15. A digit that is not allowed in the stated base makes the numeral invalid: for example, digit 8 cannot appear in an octal number.

To test whether a numeral is valid, compare every digit with the allowed range for its base. Binary permits only 0 and 1; octal permits 0 through 7; decimal permits 0 through 9; hexadecimal permits 0 through 9 and A through F. A numeral can be converted only after it passes this check. For example,  $128_8$  is not a valid octal numeral because it contains the digit 8, even though the same characters could be interpreted under a different base.

**Definition (Binary).** *A base-2 positional number system using digits 0 and 1.*

**Definition (Octal).** *A base-8 positional number system using digits 0 through 7.*

**Definition (Decimal).** *A base-10 positional number system using digits 0 through 9.*

**Definition (Hexadecimal).** *A base-16 positional number system using digits 0 through 9 and letters A through F, where A represents 10 and F represents 15.*

Each position contributes its digit multiplied by a power of the base. For  $1011_2$ , the rightmost position is  $2^0$ , the next is  $2^1$ , then  $2^2$ , and then  $2^3$ :

$$1011_2 = 1(2^3) + 0(2^2) + 1(2^1) + 1(2^0) = 8 + 0 + 2 + 1 = 11_{10}.$$

The rightmost position is the least-significant position because it has the smallest power.

|                            |           |           |           |           |
|----------------------------|-----------|-----------|-----------|-----------|
| <b>Digit</b>               | 1         | 0         | 1         | 1         |
| <b>Position from right</b> | 3         | 2         | 1         | 0         |
| <b>Place value</b>         | $2^3 = 8$ | $2^2 = 4$ | $2^1 = 2$ | $2^0 = 1$ |
| <b>Contribution</b>        | 8         | 0         | 2         | 1         |

The place-value method works in every base. Multiply each digit by the power assigned to its position, then add the contributions. The rightmost digit always has power zero; moving one place left increases the exponent by one. This is why leading zeros do not change a value: each added zero contributes zero at a new, higher place.

## 50.2 Converting decimal and binary

To convert a positive decimal integer to binary, divide repeatedly by two. At each step, keep the remainder, which must be either zero or one. Stop when the quotient becomes zero, then read the remainders from the last division upward. Reading them in the order produced gives the wrong direction.

Use this sequence every time:

1. Divide the current integer by 2.
2. Record the remainder, either 0 or 1.
3. Replace the current integer with the quotient and repeat until the quotient is 0.
4. Read the recorded remainders from last to first.
5. Verify the result by adding its binary place-value contributions.

**Example 50.2.1.** *Convert  $45_{10}$  to binary:*

$$\begin{aligned}
 45 &= 2(22) + 1 \\
 22 &= 2(11) + 0 \\
 11 &= 2(5) + 1 \\
 5 &= 2(2) + 1 \\
 2 &= 2(1) + 0 \\
 1 &= 2(0) + 1
 \end{aligned}$$

*The remainders from top to bottom are 1, 0, 1, 1, 0, 1. Reading bottom to top gives  $101101_2$ . Check by place value:  $1(32) + 0(16) + 1(8) + 1(4) + 0(2) + 1(1) = 45$ , so  $45_{10} = 101101_2$ .*

**Example 50.2.2.** *Convert  $13_{10}$  to binary. Repeated division gives  $13 = 2(6) + 1$ ,  $6 = 2(3) + 0$ ,  $3 = 2(1) + 1$ , and  $1 = 2(0) + 1$ . Reading the remainders upward gives  $1101_2$ . Check:  $1(8) + 1(4) + 0(2) + 1(1) = 13$ .*

## 50.3 Binary grouping for octal and hexadecimal

Three binary bits encode one octal digit because  $2^3 = 8$ . Four binary bits encode one hexadecimal digit because  $2^4 = 16$ . To convert a binary numeral, start at the right and group bits into threes for octal or fours for hexadecimal. If the leftmost group is short, add leading zeros; leading zeros do not change the value.

**Example 50.3.1.** Convert  $1111111_2$  to octal. Group from the right: 001 111 111. The groups are  $001_2 = 1_8$ ,  $111_2 = 7_8$ , and  $111_2 = 7_8$ , so the answer is  $177_8$ . Check:  $1(8^2) + 7(8) + 7 = 64 + 56 + 7 = 127$ , the same value as  $1111111_2$ .

**Example 50.3.2.** Convert  $1111111_2$  to hexadecimal. Group from the right: 0111 1111. These groups are 7 and 15, and hexadecimal digit 15 is F. Therefore the answer is  $7F_{16}$ . Check:  $7(16) + 15 = 112 + 15 = 127$ .

**Example 50.3.3.** Convert  $2F_{16}$  to decimal. The hexadecimal digit F is 15:

$$2F_{16} = 2(16^1) + 15(16^0) = 32 + 15 = 47_{10}.$$

## 50.4 Character codes, Unicode, and decimal digits

**Definition (ASCII).** A character-encoding standard whose original character set uses 7-bit codes.

**Definition (Unicode).** A standard that assigns code points to characters across many writing systems.

**Definition (UTF-8).** A variable-length encoding form for Unicode code points.

**Definition (Binary-Coded Decimal (BCD)).** A representation in which each decimal digit is encoded separately using a 4-bit code.

| Representation | What it identifies or encodes        | Key property                              |
|----------------|--------------------------------------|-------------------------------------------|
| ASCII          | Characters                           | Original ASCII uses 7-bit codes           |
| Unicode        | Characters from many writing systems | Assigns a code point to a character       |
| UTF-8          | Unicode code points as encoded bytes | Variable-length encoding                  |
| BCD            | Decimal digits                       | Uses a separate 4-bit code for each digit |

ASCII and Unicode concern character representation. Unicode code points identify characters; UTF-8 is one way to encode those code points as bytes. The code point and its byte encoding are different things: a character's Unicode identity does not by itself state how many bytes a particular encoding uses. UTF-8 is variable-length, so a character is not always stored using one byte. BCD has a different purpose: it represents each decimal digit separately using four bits. It is not the same as converting the entire multi-digit decimal value to binary.

**Example 50.4.1.** Represent decimal 59 in BCD. Encode the digit 5 as 0101 and the digit 9 as 1001, giving 0101 1001. Ordinary binary for the numeric value 59 is  $111011_2$ . These bit strings differ because BCD preserves the two decimal digits separately, while ordinary binary represents the value 59 as one base-2 numeral.

The grouping explains why BCD and ordinary binary use different numbers of bits. BCD uses one 4-bit group for each decimal digit, so a two-digit decimal value uses two groups. Ordinary binary instead places the entire quantity in powers of 2. When an item asks for BCD, split the decimal number into its digits before encoding; do not convert the whole number directly to binary.

## 50.5 RGB colour representation

**Definition (RGB).** *A colour model representing colour through red, green, and blue channel values.*

In a common 24-bit RGB representation, each of the three channels uses 8 bits. Therefore the three channels require  $8 + 8 + 8 = 24$  bits per pixel. Eight bits can encode  $2^8 = 256$  possible values per channel, from 0 through 255. The total number of possible channel combinations in that representation is  $256^3 = 16,777,216$ . This does not mean every image format or display uses exactly 24-bit RGB; other bit depths, alpha channels, palettes, or colour spaces are possible.

The channel count and the bit count are separate quantities. RGB has three channels: red, green, and blue. In the common representation described above, each channel has 8 bits, so one pixel uses 24 bits total. A value within one channel ranges from 0 to 255 because an 8-bit field has  $2^8 = 256$  possible patterns. To count all channel combinations, multiply the number of choices for each independent channel:

$$256 \times 256 \times 256 = 256^3 = 16,777,216.$$

This count follows only under the stated 8-bits-per-channel assumption.

**Example 50.5.1.** *In common 24-bit RGB, the red, green, and blue channels each use 8 bits. Adding the channel widths gives  $8 + 8 + 8 = 24$  bits per pixel.*

## 50.6 Logic gates and truth tables

**Definition (AND gate).** *A logic operation that outputs 1 only when every input is 1.*

**Definition (OR gate).** *A logic operation that outputs 1 when at least one input is 1.*

**Definition (XOR gate).** *A logic operation that outputs 1 when its two inputs differ.*

**Definition (NAND gate).** *The negation of AND: it outputs 0 only when every input is 1.*

**Definition (NOT gate).** *A one-input logic operation that reverses its input: 0 becomes 1 and 1 becomes 0.*

| A | B | AND | OR | XOR | NAND | NOT A |
|---|---|-----|----|-----|------|-------|
| 0 | 0 | 0   | 0  | 0   | 1    | 1     |
| 0 | 1 | 0   | 1  | 1   | 1    | 1     |
| 1 | 0 | 0   | 1  | 1   | 1    | 0     |
| 1 | 1 | 1   | 1  | 0   | 0    | 0     |

Read the truth table from the rule, not by memorizing an unlabelled row. AND requires both inputs to be 1. OR needs at least one 1. XOR checks whether the inputs differ. NAND first evaluates AND and then reverses that output. NOT has only one input and reverses it.

For two-input gates there are exactly four possible input pairs: 00, 01, 10, and 11. Evaluate every pair using the gate's rule:

- AND is 1 only on the 11 row.
- OR is 0 only on the 00 row.
- XOR is 1 on 01 and 10, where the inputs differ.

- NAND is the opposite of AND on every row.
- NOT does not use  $B$ ; it reverses  $A$  in each row.

OR and XOR are easy to confuse. For input  $A = 1$ ,  $B = 1$ , OR is 1 because at least one input is 1, but XOR is 0 because the two inputs are equal. NAND is checked only after evaluating AND and then taking the opposite result.

**Example 50.6.1.** For  $A = 1$  and  $B = 0$ , AND is 0 because both inputs are not 1; OR is 1 because one input is 1; XOR is 1 because the values differ; and NAND is 1 because it reverses the AND result.  $\text{NOT}(A)$  is 0.

**Example 50.6.2.** For  $A = 1$  and  $B = 1$ , AND is 1 and OR is 1. XOR is 0 because the inputs do not differ. NAND is the opposite of AND, so it is 0.  $\text{NOT}(A)$  is 0.

## Exercises

1. Convert  $29_{10}$  to binary and verify the result by place value.
2. Convert  $101101_2$  to octal and hexadecimal.
3. Explain the difference between BCD for decimal 25 and ordinary binary for the value 25.
4. In common 24-bit RGB, how many bits are assigned to each channel? Is that rule universal for all images?
5. For  $A = 0$ ,  $B = 1$  and for  $A = 1$ ,  $B = 1$ , find AND, OR, XOR, NAND, and  $\text{NOT}(A)$ .

## Solutions

1. Repeated division gives  $29 = 2(14) + 1$ ,  $14 = 2(7) + 0$ ,  $7 = 2(3) + 1$ ,  $3 = 2(1) + 1$ , and  $1 = 2(0) + 1$ . Reading upward gives  $11101_2$ . Check:  $16 + 8 + 4 + 0 + 1 = 29$ .
2. Group from the right: 101 101, so the octal result is  $55_8$ . Pad and group by four: 0010 1101, so the hexadecimal result is  $2D_{16}$ .
3. BCD for 25 is 0010 0101, encoding 2 and 5 separately. Ordinary binary for the value 25 is  $11001_2$ .
4. Eight bits per channel, since  $8 + 8 + 8 = 24$ . No; this describes a common 24-bit representation, not every image or display.
5. For  $A = 0$ ,  $B = 1$ : AND=0, OR=1, XOR=1, NAND=1, and  $\text{NOT}(A)$ =1. For  $A = 1$ ,  $B = 1$ : AND=1, OR=1, XOR=0, NAND=0, and  $\text{NOT}(A)$ =0.